

Matrix

矩阵

本文介绍线性代数中一个非常重要的内容——矩阵（Matrix），主要讲解矩阵的性质、运算，以及矩阵乘法的一些应用。

向量与矩阵

在线性代数中，向量分为列向量和行向量。

▼ Warning

在中国台湾地区关于「列」与「行」的翻译，恰好与中国大陆地区相反。在 **OI Wiki** 按照中国大陆地区的习惯，采用列（column）与行（row）的翻译。

线性代数的主要研究对象是列向量，约定使用粗体小写字母表示列向量。在用到大量向量与矩阵的线性代数中，不引起混淆的情况下，在手写时，字母上方的向量记号可以省略不写。

向量也是特殊的矩阵。如果想要表示行向量，需要在粗体小写字母右上方写转置记号。行向量在线性代数中一般表示方程。

引入

矩阵的引入来自于线性方程组。与向量类似，矩阵体现了一种对数据「打包处理」的思想。

例如，将线性方程组：

一般用圆括号或方括号表示矩阵。将上述系数抽出来，写成矩阵乘法的形式：

简记为：

即未知数列向量 x ，左乘一个矩阵 A ，得到列向量 b 。这个式子可以认为是线性代数的基本形式。

线性代数主要研究的运算模型是内积。内积是先相乘再相加，是行向量左乘列向量，得到一个数的过程。

矩阵乘法是内积的拓展。矩阵乘法等价于左边矩阵抽出一行，与右边矩阵抽出一列进行内积，得到结果矩阵的对应元素，口诀「左行右列」。

当研究对象是右边的列向量时，矩阵乘法相当于对列向量进行左乘。在左乘的观点下，矩阵就是对列向量的变换，将矩阵乘法中右边矩阵的每一个列向量进行变换，对应地得到结果矩阵中每一个列向量。

矩阵可以对一个列向量进行变换，也可以对一组列向量进行「打包」变换，甚至可以对整个空间——即全体列向量进行变换。当矩阵被视为对整个空间变换的时候，也就脱离了空间，成为了纯粹变换的存在。

定义

对于矩阵，主对角线是指的元素。

一般用 E 来表示单位矩阵，就是主对角线上为 1，其余位置为 0。

同型矩阵

两个矩阵，行数与列数对应相同，称为同型矩阵。

方阵

行数等于列数的矩阵称为方阵。方阵是一种特殊的矩阵。对于「 n 阶矩阵」的习惯表述，实际上讲的是 n 阶方阵。阶数相同的方阵为同型矩阵。

研究方程组、向量组、矩阵的秩的时候，使用一般的矩阵。研究特征值和特征向量、二次型的时候，使用方阵。

主对角线

方阵中行数等于列数的元素构成主对角线。

对称矩阵

如果方阵的元素关于主对角线对称，即对于任意的 i 和 j ， a_{ij} 的元素与 a_{ji} 的元素相等，则将方阵称为对称矩阵。

对角矩阵

主对角线之外的元素均为 0 的方阵称为对角矩阵，一般记作：

式中的 a_{ii} 是主对角线上的元素。

对角矩阵是对称矩阵。

如果对角矩阵的元素均为 1，称为单位矩阵，记为 E 。只要乘法可以进行，无论形状，任何矩阵乘单位矩阵仍然保持不变。

三角矩阵

如果方阵主对角线左下方的元素均为 0，称为上三角矩阵。如果方阵主对角线右上方的元素均为 0，称为下三角矩阵。

两个上（下）三角矩阵的乘积仍然是上（下）三角矩阵。如果对角线元素均非 0，则上（下）三角矩阵可逆，逆也是上（下）三角矩阵。

单位三角矩阵

如果上三角矩阵 A 的对角线全为 1，则称 A 是单位上三角矩阵。如果下三角矩阵 B 的对角线全为 1，则称 B 是单位下三角矩阵。

两个单位上（下）三角矩阵的乘积仍然是单位上（下）三角矩阵，单位上（下）三角矩阵的逆也是单位上（下）三角矩阵。

运算

矩阵的线性运算

矩阵的线性运算分为加减法与数乘，它们均为逐个元素进行。只有同型矩阵之间可以对应相加减。

矩阵的转置

矩阵的转置，就是在矩阵的右上角写上转置「 T 」记号，表示将矩阵的行与列互换。

对称矩阵转置前后保持不变。

矩阵乘法

矩阵的乘法是向量内积的推广。

矩阵相乘只有在第一个矩阵的列数和第二个矩阵的行数相同时才有意义。

设 A 为 $n \times m$ 的矩阵， B 为 $m \times k$ 的矩阵，设矩阵 C 为矩阵 A 与 B 的乘积，

其中矩阵 C 中的第 i 行第 j 列元素可以表示为：

在矩阵乘法中，结果 C 矩阵的第 i 行第 j 列的数，就是由矩阵 A 第 i 行 m 个数与矩阵 B 第 j 列 k 个数分别 **相乘再相加** 得到的。这里的 **相乘再相加**，就是向量的内积。乘积矩阵中第 i 行第 j 列的数恰好是乘数矩阵 A 第 i 个行向量与乘数矩阵 B 第 j 个列向量的内积，口诀为 **左行右列**。

线性代数研究的向量多为列向量，根据这样的对矩阵乘法的定义方法，经常研究对列向量左乘一个矩阵的左乘运算，同时也可以在这里看出「打包处理」的思想，同时处理很多个向量内积。

矩阵乘法满足结合律，不满足一般的交换律。

利用结合律，矩阵乘法可以利用 [快速幂](#) 的思想来优化。

在比赛中，由于线性递推式可以表示成矩阵乘法的形式，也通常用矩阵快速幂来求线性递推数列的某一项。

优化

首先对于比较小的矩阵，可以考虑直接手动展开循环以减小常数。

可以重新排列循环以提高空间局部性，这样的优化不会改变矩阵乘法的时间复杂度，但是会得到常数级别的提升。

```
1 // 以下文的参考代码为例
2 mat operator*(const mat& T) const {
3     mat res;
4     for (int i = 0; i < sz; ++i)
5         for (int j = 0; j < sz; ++j)
6             for (int k = 0; k < sz; ++k) {
7                 res.a[i][j] += mul(a[i][k], T.a[k][j]);
8                 res.a[i][j] %= MOD;
9             }
10    return res;
11 }
12
13 // 不如
14 mat operator*(const mat& T) const {
15     mat res;
16     int r;
17     for (int i = 0; i < sz; ++i)
18         for (int k = 0; k < sz; ++k) {
19             r = a[i][k];
20             for (int j = 0; j < sz; ++j)
21                 res.a[i][j] += T.a[k][j] * r, res.a[i][j] %= MOD;
22         }
23    return res;
24 }
```

方阵的逆

方阵 A 的逆矩阵 A^{-1} 是使得 $AA^{-1} = I$ 的矩阵。

逆矩阵不一定存在。如果存在，可以使用 [高斯消元](#) 进行求解。

方阵的行列式

行列式是方阵的一种运算。

参考代码

一般来说，可以用一个二维数组来模拟矩阵。

```
1 struct mat {
2     LL a[sz][sz];
3
4     mat() { memset(a, 0, sizeof a); }
5
6     mat operator-(const mat& T) const {
7         mat res;
8         for (int i = 0; i < sz; ++i)
9             for (int j = 0; j < sz; ++j) {
10                 res.a[i][j] = (a[i][j] - T.a[i][j]) % MOD;
11             }
12         return res;
13     }
14
15     mat operator+(const mat& T) const {
16         mat res;
17         for (int i = 0; i < sz; ++i)
18             for (int j = 0; j < sz; ++j) {
19                 res.a[i][j] = (a[i][j] + T.a[i][j]) % MOD;
20             }
21         return res;
22     }
23
24     mat operator*(const mat& T) const {
25         mat res;
26         int r;
27         for (int i = 0; i < sz; ++i)
28             for (int k = 0; k < sz; ++k) {
29                 r = a[i][k];
30                 for (int j = 0; j < sz; ++j)
31                     res.a[i][j] += T.a[k][j] * r, res.a[i][j] %= MOD;
32             }
33         return res;
34     }
35
36     mat operator^(LL x) const {
37         mat res, bas;
38         for (int i = 0; i < sz; ++i) res.a[i][i] = 1;
39         for (int i = 0; i < sz; ++i)
40             for (int j = 0; j < sz; ++j) bas.a[i][j] = a[i][j] % MOD;
41         while (x) {
42             if (x & 1) res = res * bas;
43             bas = bas * bas;
44             x >>= 1;
45         }
46         return res;
47     }
48 };
```

看待线性方程组的两种视角

看待矩阵 A，或者变换 A，有两种视角。

第一种观点：按行看，观察 A 的每一行。这样一来把 A 看作方程组。于是就有了消元法解方程的过程。

第二种观点：按列看，观察 A 的每一列。 A 本身也是由列向量构成的。此时相当于把变换 A 本身看成了列向量组，而 x 是未知数系数，思考 A 当中的这组列向量能不能配上未知数，凑出列向量 b 。

例如，文章开头的例子变为：

解方程变为研究，是否可以通过调整三个系数 x ，使得给定的三个基向量能够凑出结果的向量。

按列看比按行看更新颖。在按列看的视角下，可以研究线性无关与线性相关。

矩阵乘法的应用

矩阵加速递推

以 [斐波那契数列 \(Fibonacci Sequence\)](#) 为例。在斐波那契数列当中，，。

如果有一道题目让你求斐波那契数列第 n 项的值，最简单的方法莫过于直接递推了。但是如果 n 的范围达到了 10^9 级别，递推就不行了，此时我们可以考虑矩阵加速递推。

根据斐波那契数列 [递推公式的矩阵形式](#)：

定义初始矩阵 M 。那么， M^n 就等于 M 这个矩阵的第一行第一列元素，也就是 F_{n+1} 的第一行第一列元素。

▼ 注意

矩阵乘法不满足交换律，所以一定不能写成 $M^n = M \times M \times \dots \times M$ 的第一行第一列元素。另外，对于 $n=0$ 的情况，直接输出 F_1 即可，不需要执行矩阵快速幂。

为什么要乘上 M 矩阵的 n 次方而不是 $n+1$ 次方呢？因为 M^0 是不需要进行矩阵乘法就能求的。也就是说，如果只进行一次乘法，就已经求出 F_2 了。如果还不是很理解为什么幂是 n ，建议手算一下。

下面是求斐波那契数列第 n 项对 1000000007 取模的示例代码（核心部分）。

```

1 constexpr int mod = 1000000007;
2
3 struct Matrix {
4     int a[3][3];
5
6     Matrix() { memset(a, 0, sizeof a); }
7
8     Matrix operator*(const Matrix &b) const {
9         Matrix res;
10        for (int i = 1; i <= 2; ++i)
11            for (int j = 1; j <= 2; ++j)
12                for (int k = 1; k <= 2; ++k)
13                    res.a[i][j] = (res.a[i][j] + a[i][k] * b.a[k][j]) % mod;
14        return res;
15    }
16 } ans, base;
17
18 void init() {
19     base.a[1][1] = base.a[1][2] = base.a[2][1] = 1;
20     ans.a[1][1] = ans.a[1][2] = 1;
21 }
22
23 void qpow(int b) {
24     while (b) {
25         if (b & 1) ans = ans * base;
26         base = base * base;
27         b >>= 1;
28     }
29 }
30
31 int main() {
32     int n = read();
33     if (n <= 2) return puts("1"), 0;
34     init();
35     qpow(n - 2);
36     println(ans.a[1][1] % mod);
37 }

```

这是一个稍微复杂一些的例子。

我们发现， $dp[i][j]$ 和 $dp[i-1][j]$ 有关，于是考虑构造一个矩阵描述状态。

但是发现如果矩阵仅有这三个元素 $\begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$ 是难以构造出转移方程的，因为乘方运算和 dp 无法用矩阵描述。

于是考虑构造一个更大的矩阵。

我们希望构造一个递推矩阵可以转移到

转移矩阵即为

矩阵表达修改

▼ 「THUSCH 2017」大魔法师

大魔法师小 L 制作了一个魔力水晶球，每个水晶球有水、火、土三个属性的能量值。小 L 把这个水晶球在地上从前向后

排成一行，然后开始今天的魔法表演。

我们用 $a_{i,j}$ 分别表示从前向后第 i 个水晶球（下标从 1 开始）的水、火、土的能量值。

小 L 计划施展 k 次魔法。每次，他会选择一个区间 $[l, r]$ ，然后施展以下 k 大类、 m 种魔法之一：

1. 魔力激发：令区间里每个水晶球中 **特定属性** 的能量爆发，从而使另一个 **特定属性** 的能量增强。具体来说，有以下三种可能的表现形式：

- 火元素激发水元素能量：令 $a_{i,w}$ 增加 $a_{i,f}$ 。
- 土元素激发火元素能量：令 $a_{i,f}$ 增加 $a_{i,t}$ 。
- 水元素激发土元素能量：令 $a_{i,t}$ 增加 $a_{i,w}$ 。

需要注意的是，增强一种属性的能量并不会改变另一种属性的能量，例如 $a_{i,w}$ 并不会使 $a_{i,f}$ 增加或减少。

2. 魔力增强：小 L 挥舞法杖，消耗自身 p 点法力值，来改变区间里每个水晶球的 **特定属性** 的能量。具体来说，有以下三种可能的表现形式：

- 火元素能量定值增强：令 $a_{i,f}$ 增加 p 。
- 水元素能量翻倍增强：令 $a_{i,w}$ 增加 $a_{i,w}$ 。
- 土元素能量吸收融合：令 $a_{i,t}$ 增加 $a_{i,w}$ 。

3. 魔力释放：小 L 将区间里所有水晶球的能量聚集在一起，融合成一个新的水晶球，然后送给场外观众。生成的水晶球每种属性的能量值等于区间内所有水晶球对应能量值的代数和。需要注意的是，魔力释放的过程不会真正改变区间内水晶球的能量。

值得一提的是，小 L 制造和融合的水晶球的原材料都是定制版的 OI 工厂水晶，所以这些水晶球有一个能量阈值。当水晶球中某种属性的能量值大于等于这个阈值时，能量值会自动对阈值取模，从而避免水晶球爆炸。

小 W 为小 L（唯一的）观众，围观了整个表演，并且收到了小 L 在表演中融合的每个水晶球。小 W 想知道，这些水晶球蕴涵的三种属性的能量值分别是多少。

由于矩阵的结合律和分配律成立，单点修改可以自然地推广到区间，即推出矩阵后直接用线段树维护区间矩阵乘积即可。

下面将举几个例子。

的转移

的转移

▼ 「LibreOJ 6208」树上询问

有一棵 n 节点的树，根为 1 号节点。每个节点有两个权值 u, v ，初始值均为 1。

给出三种操作：

1. 操作：将 u 到根的路径上所有点的 v 乘以 x 。
2. 操作：将 v 到根的路径上所有点的 u 乘以 x 。
3. 操作：询问点 x 的权值 u, v 。

若直接思考，下放操作和维护信息并不是很好想。但是矩阵可以轻松地表达。

定长路径统计

▼ 问题描述

给一个 n 阶有向图，每条边的边权均为 1，然后给一个整数 k ，你的任务是对于所有点对 (i, j) 求出从 i 到 j 长度为 k 的路径的数量（不一定是简单路径，即路径上的点或者边可能走多次）。

我们将这个图用邻接矩阵 A （对于图中的边 (i, j) ，令 $A_{ij} = 1$ ，其余为 0 的矩阵；如果有重边，则设 A_{ij} 为重边的数量）表示这个有向图。下述算法同样适用于图有自环的情况。

显然，该邻接矩阵对应 $k=1$ 时的答案。

假设我们知道长度为 k 的路径条数构成的矩阵，记为矩阵 F_k ，我们想求 F_k 。显然有 DP 转移方程

我们可以把它看作矩阵乘法的运算，于是上述转移可以描述为

那么把这个递推式展开可以得到

k 次

要计算这个矩阵幂，我们可以使用快速幂（二进制取幂）的思想，在 $O(n^3 \log k)$ 的复杂度内计算结果。

定长最短路

▼ 问题描述

给你一个 n 阶加权有向图和一个整数 k 。对于每个点对 (i, j) 找到从 i 到 j 的恰好包含 k 条边的最短路的长度。（不一定是简单路径，即路径上的点或者边可能走多次）

我们仍构造这个图的邻接矩阵 A ，表示从 i 到 j 的边权。如果 i, j 两点之间没有边，那么 $A_{ij} = \infty$ 。（有重边的情况取边权的最小值）

显然上述矩阵对应 $k=1$ 时问题的答案。我们仍假设我们知道 $k-1$ 的答案，记为矩阵 F_{k-1} 。现在我们想求 F_k 的答案。显然有转移方程

事实上我们可以类比矩阵乘法，你发现上述转移只是把矩阵乘法的乘积求和变成相加取最小值，于是我们定义这个运算为 $\otimes$ ，即

于是得到

展开递推式得到

k 次

我们仍然可以用矩阵快速幂的方法计算上式，因为它显然是具有结合律的。时间复杂度 $O(n^3 \log k)$ 。

限长路径计数/最短路

上述算法只适用于边数固定的情况。然而我们可以改进算法以解决边数小于等于 k 的情况。具体地，考虑以下问题：

▼ 问题描述

给一个 n 阶有向图，边权为 w ，然后给一个整数 k ，你的任务是对于每个点对 (s, t) 找到从 s 到 t 长度小于等于 k 的路径的数量（不一定是简单路径，即路径上的点或者边可能走多次）。

我们对于每个点 s ，建立一个虚点 s' 用于记录答案，并在图中加入 (s, s') 和 (s', s) 这两条边。那么对于点对 (s, t) ，从 s 到 t 边数小于等于 k 的路径的数量，就和从 s' 到 t 边数恰好等于 k 的路径的数量相等，这是因为对于任意一条边数为 k 的路径 P ，都存在一条边数为 $k-1$ 的路径 P' 与之——对应。

对于求边数小于等于 k 的最短路，只需对每个点加一个边权为 w 的自环即可。

习题

- [洛谷 P1962 斐波那契数列](#)，即上面的例题，同题 POJ3070
- [洛谷 P1349 广义斐波那契数列](#)，矩阵需要变化一下
- [洛谷 P1939【模板】矩阵加速（数列）](#)，矩阵变成了 2×2 的矩阵，推导过程与上面差不多。

本页面部分内容译自博文 [Кратчайшие пути фиксированной длины, количества путей фиксированной длины](#) 与其英文翻译版 [Number of paths of fixed length/Shortest paths of fixed length](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

本页面最近更新：2024/10/9 22:38:42, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[Gesrua](#), [StudyingFather](#), [369Pai](#), [Anguei](#), [CCXXLI](#), [Chrogeek](#), [countercurrent-time](#), [Enter-tainer](#), [Great-designer](#), [H-J-Granger](#), [Haohu Shen](#), [Henry-ZHR](#), [InsZVA](#), [Ir1d](#), [ksyx](#), [kxccc](#), [leoleoasd](#), [Marcythm](#), [MegaOwler](#), [Menci](#), [oldherd](#), [ouuan](#), [sshwy](#), [SukkaW](#), [Tiphereth-A](#), [TrisolarisHD](#), [xcmvec](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用